

Computation of radiative fluxes challenge

Prof. Ferrandi Fabrizio - Academic Year 2023/2024

Tommaso Trabacchin

Contents

1	Introduction	1
1.1	Aims of the challenge	1
1.2	Background	1
2	Method	2
2.1	Analysis of the kernel	2
2.2	Serialization of the kernel	2
2.3	Optimization of the kernel	2
3	Results	2
3.1	Profiling procedure	2
3.2	Correctness check	3
3.3	Results for the serialized kernel	3
3.4	Results for the optimized kernel	3
4	Conclusions	3

1 Introduction

1.1 Aims of the challenge

The aims of the challenge are:

- rewrite into C serial code the `gas_optical_depth_minor_kernel`, verifying that the accuracy of the whole application does not drop significantly, and then asses the performance loss caused by the serialization of the kernel;
- find a way to improve the performance of the application, with respect to the serial baseline.

1.2 Background

The kernel to rewrite is part of a toolbox for radiation calculation for dynamic models. Its role is to compute the optical depth τ of minor species, defined as the natural logarithm of the ratio of incident to transmitted radiant power through a material.

	1 st call	2 nd call	3 rd call	4 th call	5 th call	6 th call	7 th call	8 th call
Original version	0.8	0.33	0.8	0.31	0.57	0.34	0.59	0.39
Serial version	88.96	20.441	75.37	18.94	31.25	12.73	31.72	14.66
Optimized version	0.36	0.14	0.31	0.13	0.22	0.144	0.22	0.14

Table 1: Table with the profiling results (times in ms.).

2 Method

2.1 Analysis of the kernel

The kernel consists of a for loop over the minor species. For each of them, a scaling factor is computed before updating the τ value, stored in an array, provided as parameter. The threads executing the kernel are divided in a bidimensional grid of bidimensional blocks. In particular, threads are grouped along the x axis of each block. Each group of threads is responsible for computing the tau values for a specific column-layer combination, comprising many g-points. (For each g-point, the computation is performed by a single thread). It should be noted that the computation of the scaling factor, is carried out, in each group, by the thread with `threadIdx.x == 0`. Provided that an upper bound on the number of minor species is known at compile time, it is possible to optimize the kernel, allowing each thread to compute the scaling factor for one or more minor species. This would improve the efficiency of the algorithm, as in the non optimized version, a relevant number of the threads would not carry out any useful work for an extended period of time. Also, all the values that the threads need to compute to carry out the innermost for loop could be precomputed in parallel as well.

2.2 Serialization of the kernel

The task of serializing the kernel has been quite straightforward, as the original code was rather close to a sequential version. In fact, it has been sufficient to introduce two outer for loops to mimic the multidimensional structure of the threads grid. The choice of having two nested loops has been motivated by the structure of the array containing the τ values. Only very few minor additional changes in the code were needed to complete the task. Obviously, a loss in performance is very well expected. Results of this serialization are presented in the next section.

2.3 Optimization of the kernel

As previously noted in section 2.1, the original version of the code, computed the scaling factors rather inefficiently. In order to improve performance, a change in the code structure was introduced. In this way, the first task executed by each thread is to precompute the scaling factor for one (or more) column-layer-minor combination. The values are stored in a shared array. Given that the size of shared arrays in CUDA must be known at compile time, at least an upper bound on the minors number must be known at compile time as well. This issue does not appear to be of serious concern. In addition to this, also the values computed by each thread before executing the innermost for loop are precomputed alongside the scaling factors. It is natural to expect an improvement in the performance of the kernel. Results are discussed in the next section.

3 Results

3.1 Profiling procedure

In order to assess the performance of the proposed implementations (along with the original one, as reference), their execution time have been measured, ignoring any possible overhead, such as the data transfer

from and to the GPU. All the test have been conducted using the Google Colaboratory platform, on an Nvidia T4 GPU. As a way to obtain more precise results, every implementation has been run multiple times, and their average has been reported in the next subsections. Given that during the application execution the kernel is launched eight times, in the following subsections all the eight execution times have been reported.

3.2 Correctness check

To ensure the correctness of the proposed implementations, all the outputs accuracy have been tested using a tool already available in the application repository. The outcome is that all the implementations have the same accuracy as the original one's.

3.3 Results for the serialized kernel

Results for the serialized kernel are presented in Table 1. As expected, this version is significantly slower than the original one, with an average speed down of 71.

3.4 Results for the optimized kernel

Results for the serialized kernel are presented in Table 1. It can be noted that this version can achieve better performance than the original one. The average speed up is roughly 2.5.

4 Conclusions

The main observation to be made after completing this challenge is surely that the advantage of using GPUs is terrific. In fact, the serial translation of the kernel code resulted in a dramatic speed down. However, it also shows how an efficient use of the shared memory can be crucial, in order to fully exploit the GPUs capabilities.